

SAE 5.01

Rapport de semaine 25/09/2026

Avancement au niveau du modèle :

Cette semaine nous avons commencé le développement du système de reconnaissance des voitures.

Nous avons choisi d'utiliser Python avec TensorFlow/Keras et une architecture MobileNetV3Small. Cette architecture est adaptée à la classification d'images et elle est légère pour envisager l'utilisation dans l'application mobile, de plus c'était plus facile pour avoir un début et se faire "une impression" et voir comment cette architecture fonctionne.

Un premier dataset a été préparé et organisé en différentes classes correspondant aux modèles de voitures à reconnaître. Nous avons uniquement choisi 6 voitures qui sont assez ressemblantes pour voir si le modèle arrivait bien à faire une différence. Les images ont été séparées en ensembles d'entraînements et de tests. Ceci permet de vérifier si le modèle a réellement appris à reconnaître une voiture et pas juste mémorisé les images avec lesquelles il s'est entraîné.

De la "data-augmentation" a été mise en place pour présenter différentes versions des images et permettre un meilleur apprentissage. La data-augmentation est le fait de zoomer, inverser et donner des rotations à une image.

L'entraînement a été réalisé par lots de 16 images sur 10 "epochs" (le nombre de epoch correspond au nombre de fois que le modèle a vu toutes les images). La fonction de perte "sparse_categorical_crossentropy" permet de mesurer l'erreur entre les prédictions et les classes attendues. Il y a aussi un système d'ajustement progressif des paramètres entraînaibles qui permet de réduire cette erreur (Adam) : Nous avons retrouvé la plupart de nos informations ici : <https://keras.io/api/optimizers/adam/>

Enfin un script de prédiction a été développé, il charge le modèle qui vient d'être entraîné, prépare une nouvelle image qui n'est pas connue par le modèle (ex: une photo quelconque prise sur internet d'une voiture ou, plus tard, la photo prise par l'utilisateur avec son téléphone.) et affiche les probabilités obtenues pour les différentes classes de véhicules :

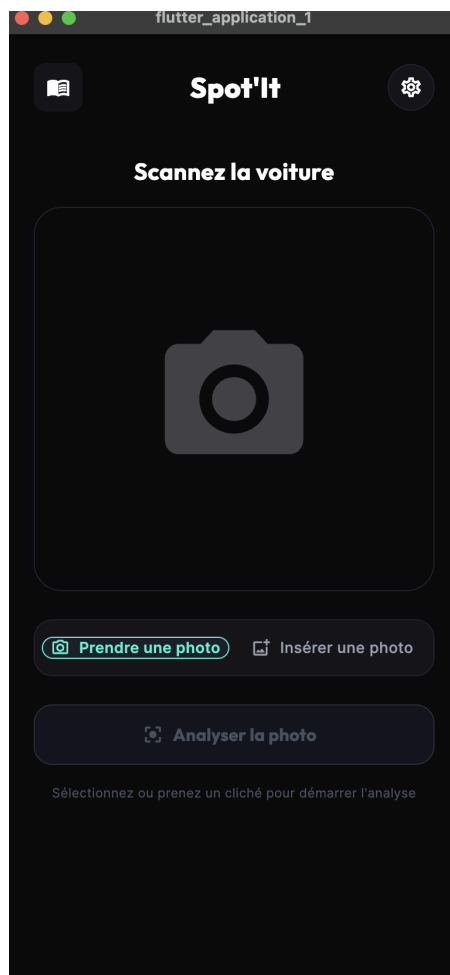
```
Voici les prédictions :  
BMW_Z4_Coupe_E86_20062009 : 46.27 %  
BMW_Z4_Roadster_E89_20092013 : 23.18 %  
MAZDA_MX5_Miata_2015Present : 14.12 %  
BMW_Z4_Roadster_LCI_E89_20132016 : 8.60 %  
BMW_Z4_G29_2018Present : 5.16 %  
BMW_Z4_M_Roadster_E85_20062009 : 2.68 %
```

Capture d'écran du premier test

Interface Flutter - Frontend

Pour le frontend, on est parti sur du Flutter. C'est le framework qu'on connaît déjà un peu grâce à nos cours, donc ça nous a permis de ne pas perdre de temps à apprendre un nouvel outil en plus de tout le reste du projet.

On a construit l'écran principal de l'application d'abord sur figma pour avoir une maquette de ce qu'on voulait, ensuite nous avons traduit cette maquette figma en flutter en suivant la structure donnée par figma. L'application permet à l'utilisateur de prendre une photo avec l'appareil photo ou en choisir une dans la galerie, puis lancer une "analyse" (pour l'instant un texte de remplacement en attendant que le modèle IA soit intégré). Il a aussi fait la page "Ma bibliothèque" qui garde une trace des voitures scannées avec l'image, le nom reconnu et la date, ainsi que la page des réglages avec un formulaire de connexion/inscription. Jusqu'à cette semaine, ce formulaire était joli mais ne faisait rien de concret comme cliquer sur "Se connecter" affichait juste l'écran de profil sans vérifier quoi que ce soit, un peu comme une maquette.



Backend et connexion à MongoDB

On a aussi repris ce formulaire de connexion pour le rendre réellement fonctionnel. Avant, comme dit plus haut, il ne vérifiait rien du tout. Le but était de faire en sorte qu'un compte créé dans l'application soit vraiment enregistré quelque part, et qu'on ne puisse se connecter qu'avec le bon mot de passe.

Première étape : créer un espace propre à l'équipe plutôt que d'utiliser un compte personnel. Un compte MongoDB Atlas a été créé sous l'adresse `spotitoff@gmail.com`, avec une base de données dédiée au projet (spotit), pour que tout le monde dans le groupe puisse s'y connecter et voir les données si besoin.

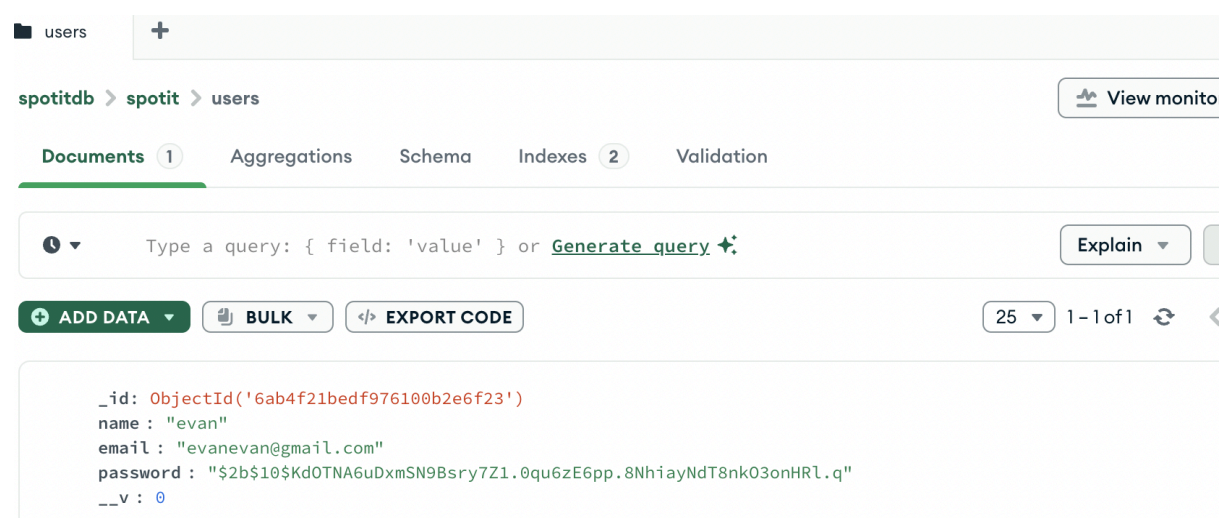
Ensuite, un petit serveur a été codé en Node.js avec Express, qui sert d'intermédiaire entre l'application et la base de données. Concrètement, il expose deux routes :

- une pour l'inscription, qui reçoit le nom/email/mot de passe, transforme le mot de passe en une version hashée (illisible, impossible à retrouver même en cas de piratage de la base) grâce à la librairie `bcrypt`, puis enregistre le tout dans MongoDB ;
- une pour la connexion, qui vérifie que l'email existe et que le mot de passe correspond, puis renvoie un token (une sorte de "ticket" temporaire, valable 7 jours, qui prouve que l'utilisateur est bien connecté).

Par exemple, un compte créé depuis l'app ressemble à ça une fois stocké dans MongoDB :

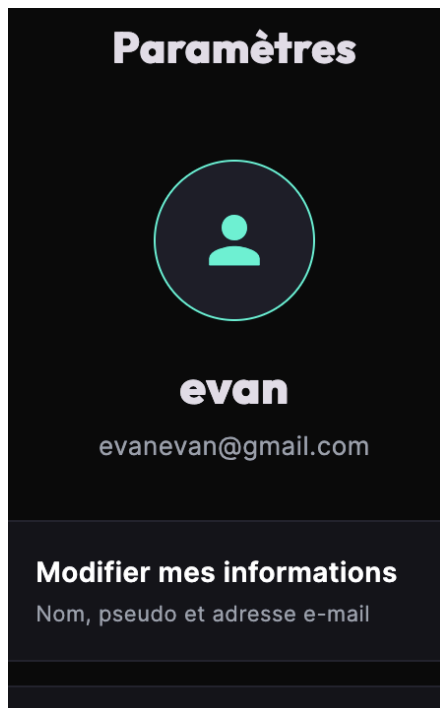
```
{
  name: "Evan",
  email: "evan@example.com",
  password:
"$2b$10$UREFwsRUoyF0CRqGNK0LzO0HM/jLhgUCNNIJ9RJAqMUQ74crIJ1Vu"
}
```

Le mot de passe n'est jamais stocké en clair, seulement ce genre de chaîne hashée.



The screenshot shows the MongoDB Atlas web interface. At the top, the breadcrumb navigation is 'spotitdb > spotit > users'. Below this, there are tabs for 'Documents' (1), 'Aggregations', 'Schema', 'Indexes' (2), and 'Validation'. The 'Documents' tab is active. A search bar contains the text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the search bar, there are buttons for 'ADD DATA', 'BULK', and 'EXPORT CODE'. On the right, there are controls for '25' items per page, '1 - 1 of 1' total items, and a refresh icon. The main area displays a single document in JSON format:

```
{
  "_id": ObjectId('6ab4f21bedf976100b2e6f23'),
  "name": "evan",
  "email": "evanevan@gmail.com",
  "password": "$2b$10$Kd0TNA6uDxmSN9Bsry7Z1.0qu6zE6pp.8NhiayNdT8nk03onHRl.q",
  "__v": 0
}
```

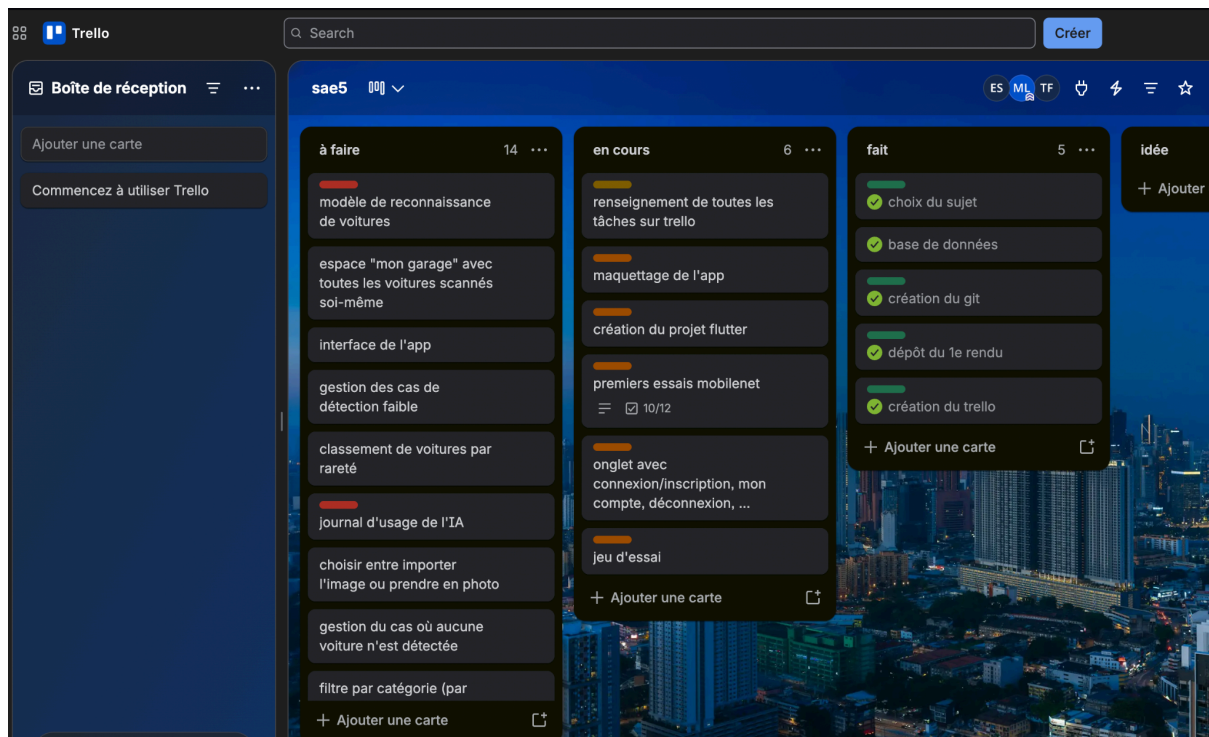


Cette partie n'a pas été toute simple à mettre en place, on a eu plusieurs difficultés sur la route comme par exemple le serveur ne trouvait pas les bonnes variables de connexion selon le dossier depuis lequel on le lançait, l'application bloquait les requêtes réseau sur Mac à cause des permissions du système (il a fallu autoriser explicitement les connexions sortantes), et on a aussi dû nettoyer notre dépôt Git parce que le dossier de dépendances Node (node_modules, plusieurs milliers de fichiers) et le fichier contenant le mot de passe de la base avaient été ajoutés par erreur avant qu'on mette en place un .gitignore.

```
evanst@MacBook-Pro-de- backend % node server.js
◇ injected env (3) from .env
MongoDB OK
Serveur sur le port 3000
```

Organisation de l'équipe

Enfin nous nous sommes occupés de la partie organisation avec la mise en place d'un tableau Trello, pour qu'on puisse tous suivre l'avancement des tâches du groupe et savoir qui fait quoi. Le tableau est disponible ici : <https://trello.com/b/TlcXJrDZ/sae5>



Traçabilité de l'usage de l'IA

Pour les fonctionnalités développées cette semaine, nous avons principalement travaillé en mode hybride, une partie du code a été écrite manuellement à partir de nos connaissances, et l'IA a été utilisée comme outil d'assistance pour gagner du temps, comprendre certaines notions nouvelles et résoudre des problèmes rencontrés pendant le développement.

Pour la partie backend avec Node.js, Express et MongoDB, le développement a été réalisé en mode hybride avec ChatGPT et Claude. Nous connaissons déjà le principe général d'une API et de la communication entre une application et un serveur, mais certaines notions comme `express.Router()`, le fonctionnement de `bcrypt`, les variables d'environnement ou encore la génération et la vérification de tokens JWT étaient nouvelles pour nous. Nous avons donc posé des questions ciblées afin de comprendre leur fonctionnement avant de les intégrer. Par exemple : « À quoi sert `express.Router()` et pourquoi l'utiliser plutôt que de mettre toutes les routes dans `server.js` ? », « Explique-moi simplement comment `bcrypt` vérifie un mot de passe alors que le hash est différent à chaque fois » ou encore « Quelle est la différence entre authentification avec session et authentification avec JWT ? ». Certaines bases de code proposées par l'IA ont été conservées après modification, tandis que d'autres propositions ont été simplifiées ou rejetées lorsqu'elles ajoutaient des éléments trop complexes ou inutiles pour notre projet.

Pour la connexion et l'inscription utilisateur, l'IA a principalement servi à accélérer des tâches que nous étions capables de réaliser nous-mêmes, par exemple la création de la structure d'une route Express ou la gestion des différentes réponses HTTP. Nous avons utilisé des prompts comme « Donne-moi un exemple minimal d'une route POST `/register` avec Express ». Nous avons ensuite adapté les réponses à notre organisation de fichiers, à nos noms de variables et à notre base MongoDB.

Pour les problèmes techniques rencontrés, l'IA a surtout été utilisée comme outil de diagnostic. Par exemple lorsque Flutter ne pouvait pas communiquer avec le serveur sur Mac, nous avons fourni le message d'erreur et les solutions proposées ont ensuite été vérifiées et testées une par une.

Pour la partie frontend Flutter, le développement a été réalisé majoritairement à la main, avec une assistance de GitHub Copilot dans Visual Studio Code. Copilot a surtout été utilisé pour l'autocomplétion de structures répétitives que nous connaissions déjà, par exemple la création d'un Container, d'un TextField, d'un ElevatedButton ou la structure générale d'un widget. Les valeurs proposées automatiquement, comme les couleurs, espacements, tailles ou actions des boutons, ont presque toujours été modifiées afin de correspondre à notre maquette Figma et au fonctionnement réel de l'application. Nous avons donc utilisé l'IA principalement pour éviter d'écrire manuellement du code répétitif, et non pour décider de la structure graphique de l'application.

Enfin, pour la partie reconnaissance d'images avec MobileNetV3Small, nous avons utilisé ChatGPT en mode hybride. L'IA nous a aidés à obtenir une première base fonctionnelle pour le chargement du dataset, la data-augmentation, l'entraînement et la prédiction. Nous avons ensuite demandé des explications précises sur chaque élément que nous ne maîtrisions pas encore, par exemple : « Pourquoi utiliser `sparse_categorical_crossentropy` ici ? », « Que change réellement `batch_size=16` ? », « Quelle est la différence entre une epoch et un batch ? » ou « À quoi sert l'optimiseur Adam et quels paramètres modifie-t-il ? ». Les scripts ont ensuite été relus, commentés et adaptés à notre dataset. Certaines propositions de l'IA ont également été écartées lorsqu'elles rendaient le modèle inutilement complexe pour nos premiers tests, par exemple l'ajout immédiat de nombreuses couches supplémentaires ou de techniques d'optimisation avancées.

Globalement, l'IA nous a donc servi de support pédagogique, d'outil de diagnostic et d'accélérateur sur les tâches répétitives. Nous avons essayé de l'utiliser principalement pour comprendre les notions que nous ne connaissions pas encore et pour gagner du temps sur celles que nous maîtrisions déjà. Les réponses n'ont pas été intégrées directement sans vérification : elles ont été testées, adaptées ou rejetées lorsqu'elles ne correspondaient pas à notre projet.

Organisation du repo github

Dans notre repo github, nous avons plusieurs branches, la branche main qui pour l'instant ne contient que le readme du projet. Nous avons aussi d'autres branches:

- backend-mangodb, qui elle contient tout le travail sur l'intégration back-end dans l'application flutter.
- flutter-dev, qui elle contient tout le travail sur l'application flutter.
- test_mobilent_python_thomas, qui est réservé pour le travail sur la reconnaissance d'images par ia.

Toutes ces branches seront merge dans la main une fois que leur tâches seront complétées et testées d'ici la fin de la deadline.